

LAB EXERCISE 2

Design and Implement a Basic URL Shortener Using Spring Boot

AIM:

To design and implement a basic URL Shortener application using Spring Boot that generates a short URL for a given long URL using a hashing technique, and redirects users to the original URL.

System Architecture

The application follows a simple client-server architecture built directly on Spring Boot's MVC layer. A client (browser or Postman) sends an HTTP request to the server. `UrlShortenerController` exposes both REST endpoints and contains the hashing and uniqueness-checking logic directly (this basic version does not split the hashing logic into a separate service class). The controller talks to `UrlMappingRepository`, a Spring Data JPA interface, which persists and retrieves URL mappings from an H2 in-memory database.

```
Client (Browser / Postman) → UrlShortenerController (REST endpoints + SHA-256 hashing + uniqueness check) → UrlMappingRepository (Spring Data JPA) → H2 Database
```

Database Table Design

A single table, `url_mapping`, stores the mapping between a short code and its original long URL. The entity does not declare explicit column constraints, so Hibernate creates both columns as a default `VARCHAR(255)` with no NOT NULL or UNIQUE constraint at the database level — uniqueness is enforced only in application code (in the controller), not by the schema itself.

Column	Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO INCREMENT	Unique row identifier
short_code	VARCHAR(255)	none (checked in code, not schema)	The generated short code (6-8 Base64 characters, derived from the SHA-256 hash of the long URL)
long_url	VARCHAR(255)	none (checked in code, not schema)	The original long URL. Note: VARCHAR(255) will truncate any long URL beyond 255 characters.

REST API Design

Method	Endpoint	Description
POST	/shorten	Accepts a JSON body { "longUrl": "..." } and returns HTTP 201 Created with a JSON body { "shortUrl": "..." } containing a unique short URL.
GET	/ {shortCode}	Looks up the short code in the database and issues an HTTP 302 redirect (Location header) to the original long URL. Returns HTTP 404 if the short code does not exist.

How the Short Code is Generated

The long URL is hashed using SHA-256. The 32-byte digest is then encoded with URL-safe Base64 (`java.util.Base64.getUrlEncoder()`, no padding), and the first 6 characters of that encoding are used as the short code — this is what produces a mixed-case alphanumeric code such as `Ay1rwc`, matching the style of the assignment's sample output (`aB12XY`). A plain hex digest, which the project's original code used, cannot produce that style of code, since hex characters are limited to 0-9 and a-f — this was the one change made to the project for this report, and nothing else in the codebase was modified. Before saving, the code checks the database for a collision: if the 6-character code is already taken, it tries 7, then 8 characters of the same Base64 encoding.

Note: unlike a more complete implementation, this controller does not first check whether the submitted long URL already has an existing mapping. So re-submitting the same long URL a second time does not return the original short code — it generates and stores a second mapping for the same long URL (see Request 3 in the Verification section). Both short codes still correctly redirect to the same long URL; the long URL is simply duplicated across two rows.

SOURCE CODE

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/>
    </parent>

    <groupId>com.ssn</groupId>
    <artifactId>url-shortener</artifactId>
    <version>1.0.0</version>
    <name>url-shortener</name>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
```

```

        <scope>runtime</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

UrlShortenerApplication.java

```

package com.ssn.urlshortener;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class UrlShortenerApplication {

    public static void main(String[] args) {
        SpringApplication.run(UrlShortenerApplication.class, args);
    }
}

```

UrlMapping.java (JPA Entity)

```

package com.ssn.urlshortener;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class UrlMapping {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String shortCode;
    private String longUrl;

    public UrlMapping() {
    }

    public UrlMapping(String shortCode, String longUrl) {
        this.shortCode = shortCode;
        this.longUrl = longUrl;
    }

    public Long getId() {
        return id;
    }

    public String getShortCode() {
        return shortCode;
    }
}

```

```

    }

    public void setShortCode(String shortCode) {
        this.shortCode = shortCode;
    }

    public String getLongUrl() {
        return longUrl;
    }

    public void setLongUrl(String longUrl) {
        this.longUrl = longUrl;
    }
}

```

UrlMappingRepository.java

```

package com.ssn.urlshortener;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UrlMappingRepository extends JpaRepository<UrlMapping, Long> {

    Optional<UrlMapping> findByShortCode(String shortCode);
}

```

ShortenRequest.java

```

package com.ssn.urlshortener;

public class ShortenRequest {

    private String longUrl;

    public String getLongUrl() {
        return longUrl;
    }

    public void setLongUrl(String longUrl) {
        this.longUrl = longUrl;
    }
}

```

ShortenResponse.java

```

package com.ssn.urlshortener;

public class ShortenResponse {

    private String shortUrl;

    public ShortenResponse(String shortUrl) {
        this.shortUrl = shortUrl;
    }

    public String getShortUrl() {
        return shortUrl;
    }

    public void setShortUrl(String shortUrl) {
        this.shortUrl = shortUrl;
    }
}

```

```
}
```

UrlShortenerController.java

```
package com.ssn.urlshortener;
```

```
import org.springframework.http.HttpHeaders;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
import java.net.URI;  
import java.nio.charset.StandardCharsets;  
import java.security.MessageDigest;  
import java.util.Base64;  
import java.util.Optional;
```

```
@RestController
```

```
public class UrlShortenerController {
```

```
    private final UrlMappingRepository repository;
```

```
    public UrlShortenerController(UrlMappingRepository repository) {  
        this.repository = repository;  
    }
```

```
    @PostMapping("/shorten")
```

```
    public ResponseEntity<ShortenResponse> shorten(@RequestBody ShortenRequest request) {
```

```
        String longUrl = request.getLongUrl();  
        String shortCode = generateUniqueShortCode(longUrl);
```

```
        UrlMapping mapping = new UrlMapping(shortCode, longUrl);  
        repository.save(mapping);
```

```
        String shortUrl = "http://localhost:8080/" + shortCode;  
        ShortenResponse response = new ShortenResponse(shortUrl);
```

```
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
```

```
    }
```

```
    @GetMapping("/{shortCode}")
```

```
    public ResponseEntity<Void> redirect(@PathVariable String shortCode) {
```

```
        Optional<UrlMapping> result = repository.findByShortCode(shortCode);
```

```
        if (result.isEmpty()) {  
            return ResponseEntity.notFound().build();  
        }
```

```
        String longUrl = result.get().getLongUrl();
```

```
        return ResponseEntity.status(HttpStatus.FOUND)  
            .header(HttpHeaders.LOCATION, longUrl)  
            .build();
```

```
    }
```

```
    private String generateUniqueShortCode(String longUrl) {
```

```

        String hash = sha256Base64(longUrl);
        int length = 6;
        String code = hash.substring(0, length);

        while (repository.findByShortCode(code).isPresent() && length < 8) {
            length++;
            code = hash.substring(0, length);
        }

        return code;
    }

    private String sha256Base64(String input) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));

            return Base64.getUrlEncoder().withoutPadding().encodeToString(hashBytes);

        } catch (Exception e) {
            throw new RuntimeException("Error generating hash", e);
        }
    }
}

```

application.properties

```

server.port=8080

# In-memory H2 database - no installation needed, resets when app stops
spring.datasource.url=jdbc:h2:mem:urlshortenerdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Lets Hibernate create the table automatically from UrlMapping.java
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Optional: view the database in a browser at http://localhost:8080/h2-console
spring.h2.console.enabled=true

```

VERIFICATION (application run with real requests)

The application was built and run locally (mvn spring-boot:run, port 8080) and tested with real HTTP requests using curl in place of Postman, reproducing the assignment's sample input exactly, and the resulting short URL was also opened directly in a web browser to confirm the redirect works end-to-end, as required by the assignment.

```

=== Request 1: POST /shorten - sample input from the assignment brief (Ex2 - URL Shortener.pdf) ===
curl -i -X POST http://localhost:8080/shorten -H "Content-Type: application/json" -d '{"longUrl": "https://www.google.com/search?q=system+design"}'

```

```

Response:
HTTP/1.1 201 Created
Content-Type: application/json
{"shortUrl":"http://localhost:8080/Ayirwc"}

```

(mixed-case alphanumeric short code, matching the style of the brief's sample output "aB12Xy")

=== Request 2: POST /shorten - a different URL ===

```
curl -X POST http://localhost:8080/shorten -H "Content-Type: application/json" -d '{"longUrl": "https://lms.ssn.edu.in/course/view.php?id=5429"}'
```

Response:

```
{"shortUrl": "http://localhost:8080/4wlmRG"}
```

=== Request 3: POST /shorten - same URL as Request 1 again ===

```
curl -X POST http://localhost:8080/shorten -H "Content-Type: application/json" -d '{"longUrl": "https://www.google.com/search?q=system+design"}'
```

Response:

```
{"shortUrl": "http://localhost:8080/AyirwcC"}
```

NOTE: this codebase does not look up an existing mapping for the same long URL before generating

a code, so re-submitting an already-shortened URL creates a second row. Since the 6-character

code "Ayirwc" was already taken (by the row from Request 1), the uniqueness loop extended the

code to 7 characters ("AyirwcC") for the new row. Both short codes correctly redirect to the

same long URL; the long URL is simply stored twice under two different short codes.

=== Request 4: GET /{shortCode} - redirect test (raw HTTP response, via curl) ===

```
curl -i http://localhost:8080/Ayirwc
```

Response:

HTTP/1.1 302

Location: https://www.google.com/search?q=system+design

Content-Length: 0

(Location header matches the brief's expected redirect target exactly)

=== Request 5: GET /{shortCode} - unknown code ===

```
curl -i http://localhost:8080/notreal1
```

Response:

HTTP/1.1 404

Content-Length: 0

=== Request 6: Web browser test (as required by the brief: "Test the application using Postman and a web browser") ===

Opened http://localhost:8080/Ayirwc directly in a browser address bar.

Result: the browser navigated straight to the Google search results page for "system design"

(the exact page the brief specifies: https://www.google.com/search?q=system+design), confirming

the redirect works end-to-end in a real browser.

TASKS PERFORMED

#	Task	Result
1	Designed the system architecture	Controller-Repository architecture with Spring Boot and Spring Data JPA (no separate service layer).

#	Task	Result
2	Designed the database table	Single url_mapping table; uniqueness of short_code is enforced in application code, not the schema.
3	Implemented POST /shorten	Generates a Base64-encoded, SHA-256-based short code and returns HTTP 201 Created.
4	Implemented GET /{shortCode}	Looks up the code and issues a 302 redirect to the original long URL.
5	Handled unknown short codes	Returns HTTP 404 for a short code that does not exist.
6	Verified with the assignment's sample input	https://www.google.com/search?q=system+design produced a mixed-case short URL (e.g. Ayirwc) matching the sample output style, and redirected to the exact original URL.
7	Tested using curl and a web browser	Confirmed the 302 redirect via curl's raw response headers, then opened the short URL directly in a browser and confirmed it landed on the Google search page.
8	Observed duplicate-URL behaviour	Re-submitting the same long URL was found to create a second mapping under an extended short code, rather than reusing the first one.

Learning Outcomes

After completing this experiment, I was able to:

- Understand the basic workflow of a URL Shortener.
- Design a simple client-server architecture using Spring Boot.
- Develop REST APIs using Spring Boot.
- Use Spring Data JPA to store and retrieve data.
- Generate unique short codes using a hashing technique.
- Implement URL redirection using HTTP status codes.
- Test REST APIs using curl (in place of Postman) and verify redirection in a web browser.
- Recognize the difference between application-level and schema-level uniqueness enforcement.